

Gaussian Mixture Models

Bilge Elitok

M.Sc. Student in Computational Biomedicine

belitok@student.uef.fi

December 20, 2024

Gaussian Mixture Model (GMM) or Mixture of Gaussians (MoG) is a parametric probabilistic unsupervised machine learning method that is utilized for tasks such as clustering, density estimation and sampling from the estimated density. GMM assumes data follows a specific probability distribution. In the context of clustering: Gaussian distribution and it distinguishes itself from partitional clustering algorithms (e.g. k-means, k-medoids) as it allowing clusters with non-spherical shapes and different densities.

The GMMs' building blocks, the Gaussian Distribution can be motivated from a variety of perspectives. From an Information Theory perspective, maximum entropy for a single variable is achieved by Gaussian distribution (Bishop, 2006). Additionally, Central Limit Theorem states distribution of the sum set of random variables becomes increasingly Gaussian as the number of sets in the sum increases (Walker, 1969).

GMM assumes the overall data distribution is a superposition of Gaussian components where each Gaussian component represents a cluster ($k \in K = \{k_1, \dots, k_m\}$) in data and mixture proportions (π_k) sum to one, i.e. satisfying $\sum_k \pi_k = 1$. As real-world data often originates from a combination of underlying generative processes, mixture models that combine several distributions provides a framework to capture this complexity.

However, for any data point x_i , the specific Gaussian distribution(s) that are responsible of its generation are not observed. To address this, the identity of Gaussian distribution responsible for generating x_i is introduced as a latent variable z_i . Therefore, Gaussian Mixture Models involve three types of drivers for the generative process of x_i :

- **Regular Variables:** Observed variables that are drawn from a distribution (e.g. observed data point x_i)
- **Latent Variables:** Unobserved variables that are also drawn from a distribution and explain the underlying structure give rise to the observed variables. (e.g. for a data point x_i , the latent variable z_i determines if x_i is generated by a component k)
- **Parameters:** Unobserved quantities that do not have a distribution and define the model (e.g. μ_k , and the other properties of a cluster in context of clustering)

Therefore GMM models the generative process for x_i as an interaction between the parameters and the latent variable space, which provides a probabilistic framework to explain the observed data in the basis of underlying distributions. Before discovering the latent variable representation and probabilistic nature of GMM, we revisit the properties of Gaussian distributions, which work as the building blocks of Gaussian mixture models.

Properties of Univariate and Multivariate Gaussian Distributions:

Probability Distribution Function (PDF) gives the likelihood of observing a particular value of the random variable. In case of the single variable x , probability density function takes the form (Bishop, 2006):

$$P(x | \mu, \sigma^2) = \frac{1}{\sqrt{2\pi\sigma^2}} \exp\left(-\frac{(x - \mu)^2}{2\sigma^2}\right) \quad (1)$$

Where x is the random variable, μ is the distribution mean which positions the center of the Gaussian distribution, and σ^2 is the variance which controls the spreadedness. The graph of the PDF corresponds to a bell-shaped curve symmetric around its mean. Univariate Gaussian PDF assigns higher probability density to the values closer to the

mean with the peak probability density occurs for $x = \mu$.

For a D-dimensional vector x the PDF for the Multivariate Gaussian Distribution takes the following form(Bishop, 2006):

$$P(\mathbf{x} | \mu, \Sigma) = \frac{1}{\sqrt{(2\pi)^d |\Sigma|}} \exp \left(-\frac{1}{2} (\mathbf{x} - \mu)^\top \Sigma^{-1} (\mathbf{x} - \mu) \right), \quad (2)$$

Parameters of Multivariate Gaussian PDF:

- $\mathbf{x}$ is the $d \times 1$ random vector, representing the data point in d -dimensional space.
- μ is the $d \times 1$ mean vector.
- Σ is the $d \times d$ covariance matrix.
- $|\Sigma|$ denotes the determinant of Σ , representing the scaling factor for the distribution's spread in multidimensional space.
- The term $(\mathbf{x} - \mu)^\top \Sigma^{-1} (\mathbf{x} - \mu)$ is the Mahalanobis Distance, which is a generalized distance measure in multivariate space that accounts for correlation of dimensions and scaling.

The multivariate Gaussian distribution is compactly represented as $\mathbf{N}(\mu, \Sigma)$, meaning that the distribution is parametrized by the mean vector and the covariance matrix. Eigenvectors of Σ define the variance along each of the principal axes of the distribution (Bishop, 2006). Covariance matrix Σ is composed of covariances (off-diagonal elements), and of variances (diagonal elements). When different dimensions of random vector x behaves independently, off-diagonal elements becomes zero. Therefore, when covariance matrix is diagonal, it indicates dimensions are uncorrelated, independent variables do not expect to influence each other, Mahalanobis distance reduces to sum of squared distances weighted by the variance, and PDF simplifies significantly.

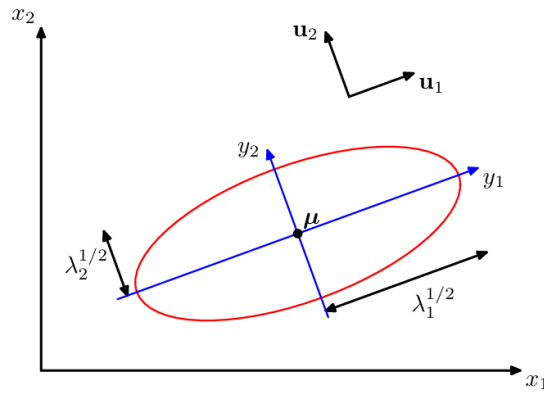


Figure 1: Representation of a Gaussian distribution in 2D space with red curve shows the constant probability density for $x = x_1, x_2$. Eigenvectors u_i of the covariance matrix, and the corresponding eigenvalues λ_i defines the major axes of the ellipse.(Bishop, 2006, pp.78-81)

As an example, the PDF of bivariate Gaussian distribution for two-dimensional random vector $\mathbf{x}$ takes the form:

$$p(x_1, x_2) = \frac{1}{2\pi |\Sigma|^{1/2}} \exp \left(-\frac{1}{2} \begin{pmatrix} x_1 - \mu_1 \\ x_2 - \mu_2 \end{pmatrix}^\top \Sigma^{-1} \begin{pmatrix} x_1 - \mu_1 \\ x_2 - \mu_2 \end{pmatrix} \right)$$

Where the 2d random vector $\mathbf{x} = \begin{bmatrix} x_1 \\ x_2 \end{bmatrix}$ Has a Gaussian Distribution where: $\mu = \begin{bmatrix} \mu_1 \\ \mu_2 \end{bmatrix}$ and the covariance matrix:

$$\Sigma = \begin{bmatrix} \Sigma_{11} & \Sigma_{12} \\ \Sigma_{21} & \Sigma_{22} \end{bmatrix}$$

Where Σ_{11} indicates variance of x_1 , Σ_{22} indicates variance of x_2 , and $\Sigma_{12}\Sigma_{21}$ indicates the covariance between x_1 and x_2 .

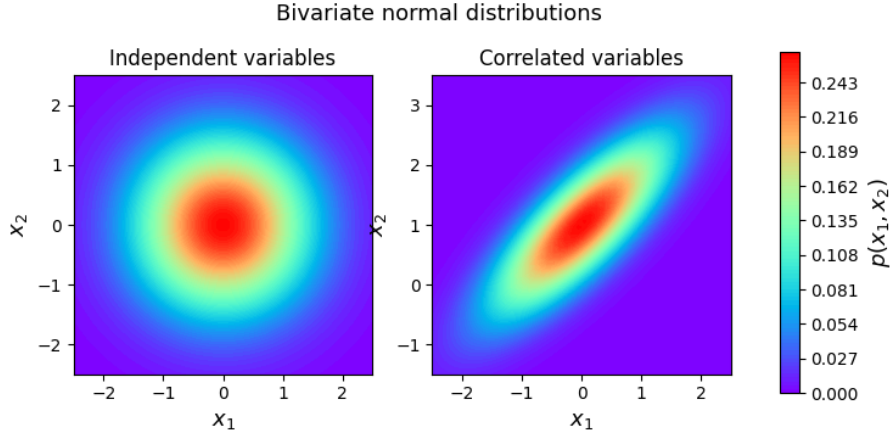


Figure 2: Bivariate Gaussian Distribution Plot for two cases: Left, $cov(x_1, x_2) = 0$. Right, covariance between x_1 and x_2 is set to 0.8. Figure by Mustafa Murat Arat from [mmuratarat.github.io](https://github.com/mmuratarat).

From Gaussian Distribution to a Probabilistic Mixture Framework:

This section is based on Pattern Recognition and Machine Learning ch.9 (Bishop, 2006)

GMM assumes the probabilistic generative processes are responsible for the observed data points which follow Gaussian distribution. Observed data is assumed to be given rise by the combination of Gaussian Distributions while each of those distributions represent a single component within the mixture, characterized by its own cluster mean (μ_k), and covariance (Σ_k), describing the shape and spread of the cluster. The relative contribution of each component is specified by the weights of the mixture (π_k) as these components collectively define the overall data distribution.

The general form of GMM appears as a weighted sum of several Gaussian distributions, where mixture weights (π_k) indicate the likelihood of a randomly chosen datapoint to be drawn from a particular Gaussian component (Barber, 2012).

$$p(x) = \sum_{k=1}^K \pi_k N(x | \mu_k, \Sigma_k) \quad (3)$$

Where:

- $p(x)$ is the probability density function (PDF) of the mixture model.
- π_k is the weight or mixing proportion of the k -th Gaussian component, with $\sum_{k=1}^K \pi_k = 1$ and $\pi_k \geq 0$ for all k .
- $N(x | \mu_k, \Sigma_k)$ is the multivariate normal distribution of the k -th component, with mean vector μ_k and covariance matrix Σ_k .
- K is the number of Gaussian components (clusters) in the mixture.

Here, the mixture weights μ_k represents a prior probability of x_i being generated by the k -th component. This prior reflects the likelihood of a data point belonging to specific Gaussian component, before any of the observations was made. This prior probability was represented as model parameter π_k , which are learned during the fitting process.

Multiple Membership of a Data Point

Unlike the hard membership concept, where each data point is limited to originate from one single cluster, GMM allows a data point belong to more than one component, with a certain degree of membership. Specifically, the degree of membership of x_i to component k is called as responsibility and denoted as γ_{ik} . The responsibility or the degree of membership based on both mixture weights and the likelihood of the data point to be observed under k :

$$\gamma_{ik} = \frac{\pi_k N(\mathbf{x}_i | \mu_k, \Sigma_k)}{\sum_{j=1}^K \pi_j N(\mathbf{x}_i | \mu_j, \Sigma_j)} \quad (4)$$

where the denominator is a normalizing term composed of the sum of all responsibilities across all components.

The probabilistic membership allows for a soft assignment and for partial membership to different components with varying degrees of responsibility. These responsibilities are also used in updating the parameters for the model fitting process. In the following sections, we will explore them further where the Expectation Maximization algorithm incorporates the responsibilities to update and optimize the model parameters (equations: 11, 13, 14, 15).

Latent Variable z_i : Unobserved Component Assignments for each x_i

In GMM, the probability of the contribution from a specific Gaussian component is modeled as the latent variable z_i . For K Gaussian components, z_i is a latent a K -dimensional vector represents the component assignment, in other words it indicates whether the corresponding Gaussian component is responsible creating x_i . z_i is typically a one-hot encoded vector, where the assigned component is set to 1, and all the rest are set to zero.

$$z_{i,k} = \begin{cases} 1 & \text{if } x_i \text{ is generated by component } k, \\ 0 & \text{otherwise.} \end{cases}$$

The joint probability of x_i and its latent variable z_i is represented as:

$$p(x_i, z_i) = p(z_i) p(x_i | z_i), \quad (5)$$

where:

- $p(z_i)$ is the prior probability of a k -th Gaussian component to be randomly selected at the first place,
- $p(x_i | z_i)$ is the likelihood of observing x_i given that it was generated by the k -th Gaussian component, which is characterized by $N(x_i | \mu_k, \Sigma_k)$.

The final form of the probability density function for a mixture of Gaussians on equation (3), is derived by marginalizing over all possible values of z_i :

$$P(x_i) = \sum_{k=1}^K P(x_i, z_{i,k}) = \sum_{k=1}^K P(z_{i,k}) P(x_i | z_{i,k}),$$

Here, the mixture weight for the k -th component is derived by marginalizing over z_i , which renders $p(z_{i,k}) = \pi_k$.

When given $z_i, k = 1$, the data point assumed to generated by k -th Gaussian component and it follows a Gaussian distribution, the conditional probability $p(x_i | z_{i,k})$ renders equal to the density of the Gaussian distribution for the k -th component so that: $p(x_i | z_{i,k}) = N(x_i | \mu_k, \Sigma_k)$.

$p(z_i = k)$, reflects our belief about how likely it is that any given data point belongs to the k -th Gaussian component before observing the data point itself. Although z_i is marginalized out via this summation over k , its role become more important as it is used to compute posterior probability for the component memberships of x_i .

Estimating the Parameters of the Model: Maximum Likelihood Estimation

This section is based on corresponding Pattern Recognition and Machine Learning chapters: ch.2 and ch.9 (Bishop, 2006)

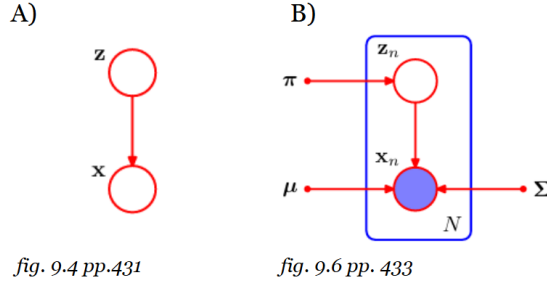


Figure 3: Graphical representation of generative process for an observed variable x . A) The joint distribution on the *equation(2)* is expressed in the graph form, and B) the graph representation for a Mixture of Gaussians (with the PDF expressed in *equation (1)*) for a set of N data points x_n with corresponding latent variables z_n . (Bishop, 2006)

For real-world data, often true parameters of the underlying model is not observed directly, the data itself can be used to estimate parameters. One intuitive understanding of this idea could be: the likelihood of the observed data will be often different for a different set of parameters. In many applications of Gaussian Mixture Models, model parameters are determined using maximum likelihood estimation (MLE). MLE is a statistical principle of finding the set of parameters that makes observations most probable under the model, i.e. maximizes the likelihood of the observed data X given the parameter set θ (Bishop, 2006).

$$L(\theta) = P(X | \theta) \quad (6)$$

The objective of maximum likelihood estimation is to find the set of parameters θ that maximizes the likelihood for the observed data X (Barber, 2012). In the objective notation below, the optimal set of parameters that maximizes the likelihood function is denoted by θ' and in other words, θ' is the solution for this optimization problem:

$$\theta' = \arg \max_{\theta} L(\theta) = \arg \max_{\theta} P(X | \theta), \quad (7)$$

Maximization of likelihood is equivalent to the maximizing the log-likelihood (Barber, 2012):

$$\log L(\theta) = \sum_1^n \log P(x_i | \theta) = \prod_i^n \log P(x_i | \theta) \quad (8)$$

The assumption of likelihood function makes the product notation (5) possible is: the **observations are independent and identically distributed (i.i.d.)** under the model. This assumption renders data points to either have the same probability mass function (for discrete data points) or the same probability density function (for continuous data points). For simplicity and consistency across both discrete and continuous cases, the shared notation $P(x_i | \theta)$ is used.

Suppose we wish to model a set of observations $X = x_1, \dots, x_N$ via a mixture of Gaussians parametrized by π, μ and Σ . In this case, the parameter vector would be $\theta = \pi, \mu, \Sigma$. The likelihood function on equation (4) for the GMM model will be:

$$L(\pi, \mu, \Sigma) = P(X | \pi, \mu, \Sigma) \quad (9)$$

Where,

- k represents the indices for the mixture components for $k \in K$, where $K = \{1, 2, \dots, K\}$,
- n represents the indices for the data points for $n \in N$, where $N = \{1, 2, \dots, N\}$,
- π_k is the mixing coefficient for each component satisfying $\sum_k^K \pi_k = 1$, it represents the prior probability of a data point generated by the k -th component.
- $N(x_n | \mu_k, \Sigma_k)$ is the Gaussian density evaluated at x_n for the component k ,

And the log likelihood function to be maximized is given by:

$$\log p(X | \pi, \mu, \Sigma) = \sum_{n=1}^N \log \left(\sum_{k=1}^K \pi_k N(x_n | \mu_k, \Sigma_k) \right). \quad (10)$$

Here, the term

$$\sum_{k=1}^K \pi_k N(x_n | \mu_k, \Sigma_k)$$

indicating each probability density $N(x_n | \mu_k, \Sigma_k)$ is weighted by the corresponding mixing coefficient π_k . In order to find the conditions that are satisfied at the maximum of the likelihood function, it is necessary to optimize over π_k, μ_k and Σ_k .

Limitation of Maximum Likelihood Estimation: Latent Component Assignments

The optimization task in equation (7) involves learning multiple parameters during the fitting process. In the presence of latent variables, this task becomes challenging due to summation inside the logarithm (equation (10)): For each data point x_i , its component assignment z_i is unobserved. This renders the overall likelihood of observed data to be dependent on latent variables. In the presence of latent variables, the marginal likelihood then requires to be completed by summing (for discrete latent variables) or integrating (for continuous latent variables) over all possible configurations of the latent variables:

$$p(\mathbf{x}) = \sum_{\mathbf{z}} p(\mathbf{x}, \mathbf{z}) \quad \text{or} \quad p(\mathbf{x}) = \int p(\mathbf{x}, \mathbf{z}) d\mathbf{z} \quad (11)$$

This marginalization introduces significant complexity and computational infeasibility to maximization of likelihood. For a latent space of higher dimensions, possible configurations grow exponentially. In other words, direct application of MLE is only tractable when exact assignment of clusters z_i for estimation of the parameters that maximizes the likelihood of observations is known.

In presence of latent variables, this optimization problem is made tractable through approximation rather than calculation. One approach to address this challenge is to iteratively estimate the model parameters and latent variables by Expectation-Maximization (EM) algorithm.

Expectation Maximization Algorithm (EM)

This section is based on corresponding Pattern Recognition and Machine Learning chapters: ch.9 (Bishop, 2006)

The Expectation Maximization Algorithm is an alternating optimization algorithm to obtain maximum likelihood estimates in presence of latent variables. In latent variable models, EM algorithm cycles between two steps to maximize the likelihood function:

First Mode: Expectation (E)-Step:

E-Step begins by the calculation of expected value of the complete likelihood function given the current parameter estimates $\theta_{current}$ and observed (X) variables over all possible values of latent variable Z . Direct calculation of often intractable integral over Z (equation (7)) is avoided by approximation using the posterior probabilities (also known as responsibilities denoted by γ) of latent variables.

- The posterior probability of Z is computed using the current parameter estimates and the responsibilities $\gamma(z_i | x_i, \theta_{current})$ for each data point and each latent variable (equation (4)).
- These responsibilities are expected values of latent variables and they represent soft cluster assignments for each data points across different mixture components.

By that, the E-Step replaces the intractable integration with a summation over latent variable states, weighted by the responsibilities. This step essentially computes the **Q-function** which represents expected value of complete-data log-likelihood given observed data and initial estimates of model parameters $\theta_{current}$:

$$Q(\theta_{new} | \theta_{current}) = \sum_{i=1}^N \sum_{k=1}^K \gamma(z_{i,k} | x_i, \theta_{current}) \log p(x_i, z_{i,k} | \theta_{new}) \quad (12)$$

Where:

- $\gamma(z_{i,k} | x_i, \theta_{current})$ is responsibility of the k -th Gaussian component in generation of the data point x_i given the current parameter estimates $\theta_{current}$.
- $\log p(x_i, z_{i,k} | \theta_{new})$ is the log of the joint probability of the data point x_i and the latent variable assignment $z_{i,k}$ under the new parameter estimates θ_{new} .

Upon finalization of E-Step the posterior probabilities $\gamma(z_{i,k} | x_i, \theta_{current})$ are soft component assignment for data points, and the joint probability represents likelihood of observations, latent variables and model parameters. Next, the Q-function is aimed to be maximized on the M-Step.

Second Mode: Maximization (M)-Step

In M-Step, the goal is to find the new parameters θ_{new} that maximize the expectation of complete-data log-likelihood calculated during the E-step. In other words this step aims for the optimization of initial parameters so that θ_{new} maximizes the Q function with respect to updated responsibilities $\gamma(z_{i,k} | x_i, \theta_{current})$. Formally, the updated parameters are obtained by solving:

$$\theta_{new} = \arg \max_{\theta} Q(\theta | \theta_{current}) \quad (13)$$

Each parameter of the model is updated and updated parameters, θ_{new} , is used to compute posterior probabilities for E-step during the next iteration. Iteration continues until the convergence is reached i.e. change in the log-likelihood between iterations becomes smaller than a threshold or if a pre-assigned number of maximum iterations is completed.

The updated model parameters that are refined during M-step by:

$$\pi_k^{new} = \frac{1}{N} \sum_{i=1}^N \gamma(z_{i,k} | x_i, \theta_{current}) \quad (14)$$

$$\mu_k^{new} = \frac{\sum_{i=1}^N \gamma(z_{i,k} | x_i, \theta_{current}) x_i}{\sum_{i=1}^N \gamma(z_{i,k} | x_i, \theta_{current})} \quad (15)$$

$$\Sigma_k^{new} = \frac{\sum_{i=1}^N \gamma(z_{i,k} | x_i, \theta_{current}) (x_i - \mu_k^{new})(x_i - \mu_k^{new})^T}{\sum_{i=1}^N \gamma(z_{i,k} | x_i, \theta_{current})} \quad (16)$$

For a more complete picture of EM optimization, we can take a look at an example where a working EM algorithm on a mixture of two Gaussians is illustrated.

Iteration of EM algorithm is illustrated in figure 1, where data is first shown as green, and the initial configuration of the mixture model where two Gaussian components are shown as blue and red circles (a). Here, we see that both cluster means, or covariances are appear raw, perfect circles randomly initialized without decentralized means. Similarly, covariance of the clusters are to-be-updated, as they appear intact, perfect circles of equal spread of any direction.

Next plot, (b), shows the final result of the E-step where each data point is color-coded with proportion to posterior probabilities of being generated by either a blue or a red component. Here, points with a significant probability for belonging to either cluster appears purple.

On plot (c), we see the situation after the first M-step, where the updated cluster means now appear at the center of the dataset. Blue Gaussian component is positioned to the center of mass of the blue ink, and red component is positioned to the overall center of red ink.

Next three plots show the results after 2, 5 and 20 complete EM cycles while in plot (f), the algorithm appear close to convergence.

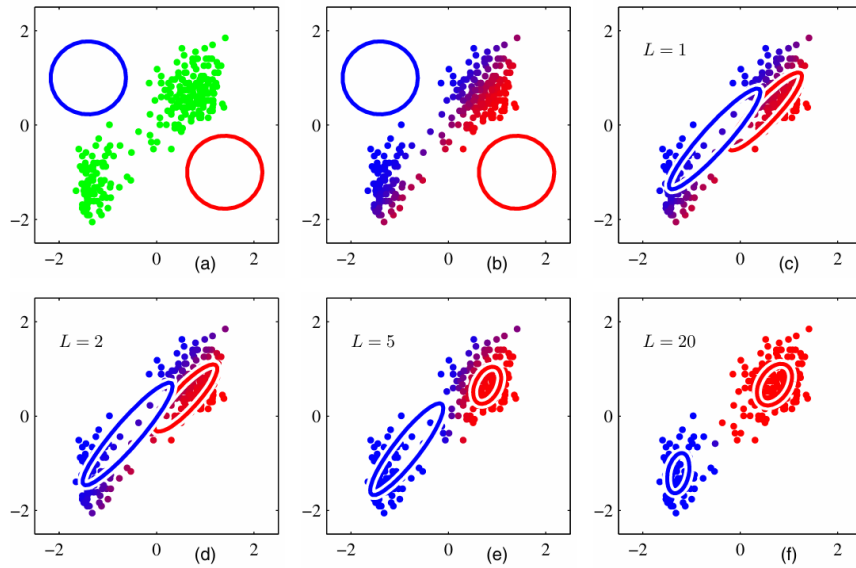


Figure 4: Illustration of EM Algorithm for a Mixture of two Gaussians (Bishop, 2006, pp.437)

Limitations for Gaussian Mixture Models

The challenges of EM algorithm directly impacts performance of GMMs in practice.

1. EM's reliance on initialization parameters renders GMMs sensitive to initialization while poor initializations can lead to converge suboptimal solutions, leading misinterpretation of the data by GMM. In order to overcome this challenge, it is common to run K-means algorithm to find suitable initialization for GMM covariance matrices and mixing coefficients. This practice is aimed to initialize GMM more convenient to the data and reduce the number of EM iterations to approximate convergence. Another practice, although can be computationally expensive, is running GMMs with multiple different initializations. (Bishop, 2006)
2. **Singularity of the Likelihood Function when GMM collapses onto a single data point:** If a component becomes perfectly aligned with a single data point, likelihood function becomes singular. Often, regularization techniques to prevent this collapse is applied.
3. **EM can get stuck in a local maximum:** Log likelihood function can have multiple local maxima in which, iterative EM optimization can get trapped. While the global maximum (largest possible log-likelihood) is not guaranteed to be found, a strategy to overcome this is running EM multiple times with different initializations.
4. **Curse of Complex Models: Overfitting** The EM algorithm might overestimate the number of components to maximize the likelihood so that it starts to fit to the noise of the data and mislearn the real structure. In order to overcome overfitting and make sure it generalizes well, cross-validation, regularization constraints or Bayesian Information Criterion (BIC) can be used.

Evaluating the Performance of Gaussian Mixture Model:

This section is based on corresponding section on scikitlearn documentation: Section 2.3.11. Clustering performance evaluation

Performance of GMM can be assigned by multiple evaluation frameworks, such as,

1. Adjusted Rand Index (ARI):

Adjusted Rand Index measures the similarity of the two assignments while accounting for the possibility of random cluster assignments. ARI ranges from -1, indicating the complete disagreement to +1, indicating the perfect agreement while a 0 ARI value means random clustering.

$$ARI = \frac{RI - \mathbb{E}[RI]}{\max(RI) - \mathbb{E}[RI]} \quad (17)$$

2. Normalized Mutual Information (NMI):

Normalized Mutual Information: measures the amount of information (agreement) shared between the predicted clusters and the true labels. A 0 NMI value indicates no mutual information, and a value of 1 indicates perfect agreement between the predicted and true labels.

$$NMI(X, Y) = \frac{I(X, Y)}{\sqrt{H(X)H(Y)}} \quad (18)$$

Where:

- $I(X, Y)$ being mutual information shared by two clusters
- $H(X)$ is the entropies of the predicted clusters
- and $H(Y)$ are the entropies of the true labels.

3. Silhouette Coefficient:

Silhouette Coefficient: measures how similar a data point is, to its own cluster, compared to any other cluster. each individual point is. It combines Cohesion and Separation.

$$S(i) = \frac{b(i) - a(i)}{\max(a(i), b(i))} \quad (19)$$

- **For Cohesion:** the distance $a(i)$ indicates how far is the average distance from a point i to all other points in its own cluster.
- **For Separation:** the distance $b(i)$ indicates how far the smallest average distance from a point i to all points in any other cluster.

Just like ARI, Silhouette Coefficient also ranges from -1 (poor clustering quality) to +1 (high clustering quality) while a 0 value indicates overlapping clusters. A poor clustering score suggests, low cohesion, meaning that the data point is not very similar to the others that share the same cluster. And it suggests high separation as well, meaning that actually, the data point is more closer to the other points in other clusters than to the ones of its own.

References

1. C. M. Bishop, *Pattern Recognition and Machine Learning*, Springer, 2006.
2. Mustafa Murat Aratat - Blog Entry: Univariate and Multivariate Gaussian Distributions
<https://mmuratarat.github.io/about/>
3. David Barber, *Bayesian Reasoning and Machine Learning*, Chapter 20.3: The Gaussian Mixture Model, pp. 370-377.
4. M. Usama et al., "Unsupervised Machine Learning for Networking: Techniques, Applications and Research Challenges," in *IEEE Access*, vol. 7, pp. 65579-65615, 2019, doi: 10.1109/ACCESS.2019.2916648.
5. Ng, A. Y., & Jordan, M. I. (2001). On discriminative vs. generative classifiers: A comparison of logistic regression and naive Bayes. *Advances in Neural Information Processing Systems*, 14, 841-848.
6. Blei, D. M. (2015). Introduction to Foundations of Graphical Models (Lecture notes). Columbia University. Retrieved from <https://www.cs.columbia.edu/~blei/fogm/2015F/notes/intro.pdf>
7. Blei, D. (2014). Build, compute, critique, repeat: Data analysis with latent variable models. *Annual Review of Statistics and Its Application*, 1:203-232.
8. Haugh, M. (2015). IEOR E4570: Machine Learning for OR&FE [Lecture slides]. Columbia University.

9. James, G., Witten, D., Hastie, T., & Tibshirani, R. (2023). An introduction to statistical learning: With applications in Python. Springer. <https://doi.org/10.1007/978-3-031-17978-6>
10. Scikit-learn Documentation. *Clustering: Clustering evaluation*, <https://scikit-learn.org/stable/modules/clustering.html#clustering-evaluation>, accessed: December 2024.